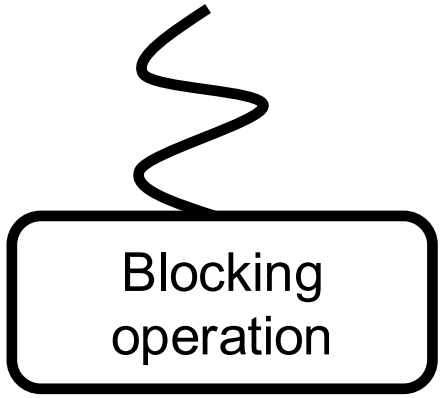
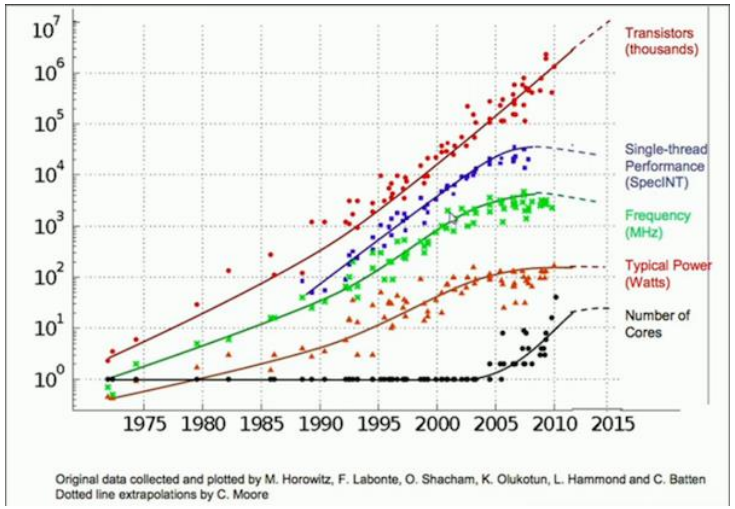
Concurrency



Thread





- A **critical section** is a piece of code that accesses a *shared* resource, usually a variable or data structure.
- A **race condition** (or **data race** [NM92]) arises if multiple threads of execution enter the critical section at roughly the same time; both attempt to update the shared data structure, leading to a surprising (and perhaps undesirable) outcome.
- An **indeterminate** program consists of one or more race conditions; the output of the program varies from run to run, depending on which threads ran when. The outcome is thus not **deterministic**, something we usually expect from computer systems.
- To avoid these problems, threads should use some kind of **mutual exclusion** primitives; doing so guarantees that only a single thread ever enters a critical section, thus avoiding races, and resulting in deterministic program outputs.

Monitors Locks Semaphores
Condition Variables

Loads Stores Test&Set
Disable Interrupts

Review: Locks

Goal: Provide mutual exclusion (mutex)

Three common operations:

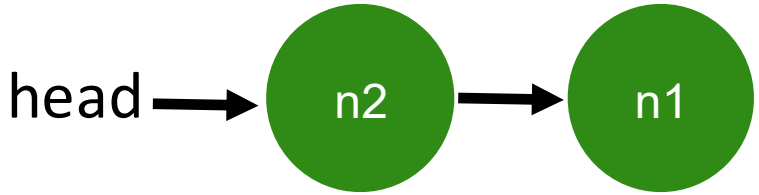
- Allocate and Initialize
  - `Pthread_mutex_t mylock = PTHREAD_MUTEX_INITIALIZER;`
- Acquire
  - Acquire exclusion access to lock;
  - Wait if lock is not available (some other process in critical section)
  - Spin or block (relinquish CPU) while waiting
  - `Pthread_mutex_lock(&mylock);`
- Release
  - Release exclusive access to lock; let another process enter critical section
  - `Pthread_mutex_unlock(&mylock);`

Shared Linked List

```
typedef struct __node_t {
        int key;
        struct __node_t *next;
} node_t;

Typedef struct __list_t {
        node_t *head;
} list_t;

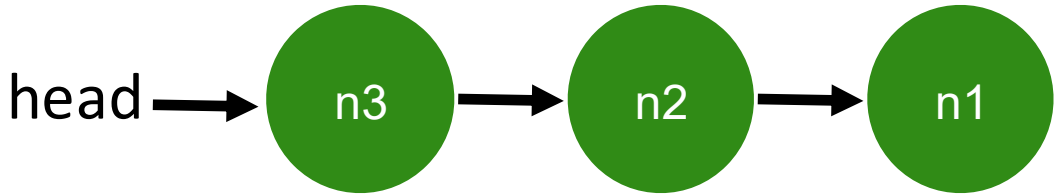
Void List_Init(list_t *L) {
        L->head = NULL;
}
```



Shared Linked List

```
Void List_Insert(list_t *L, int key) {
        node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L,  int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)
                        return  1;
                tmp = tmp->next;
        }
return 0;
}
```

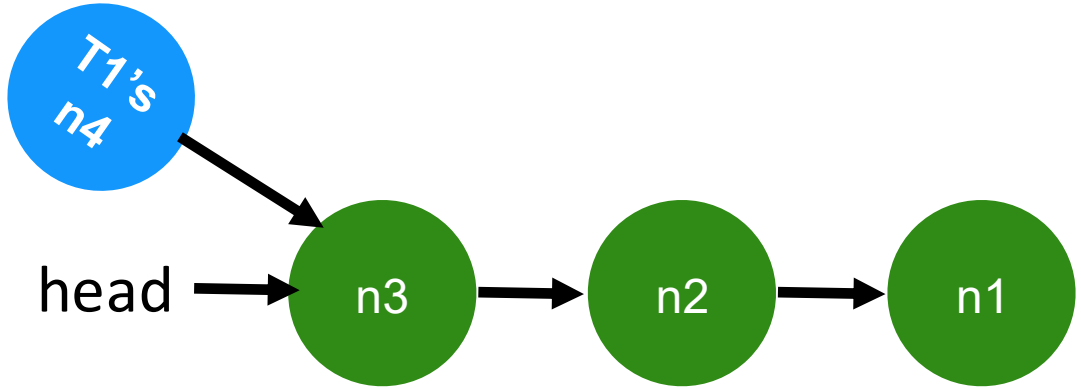


What can go wrong?
Find a schedule that leads to problem?

Linked-List Race

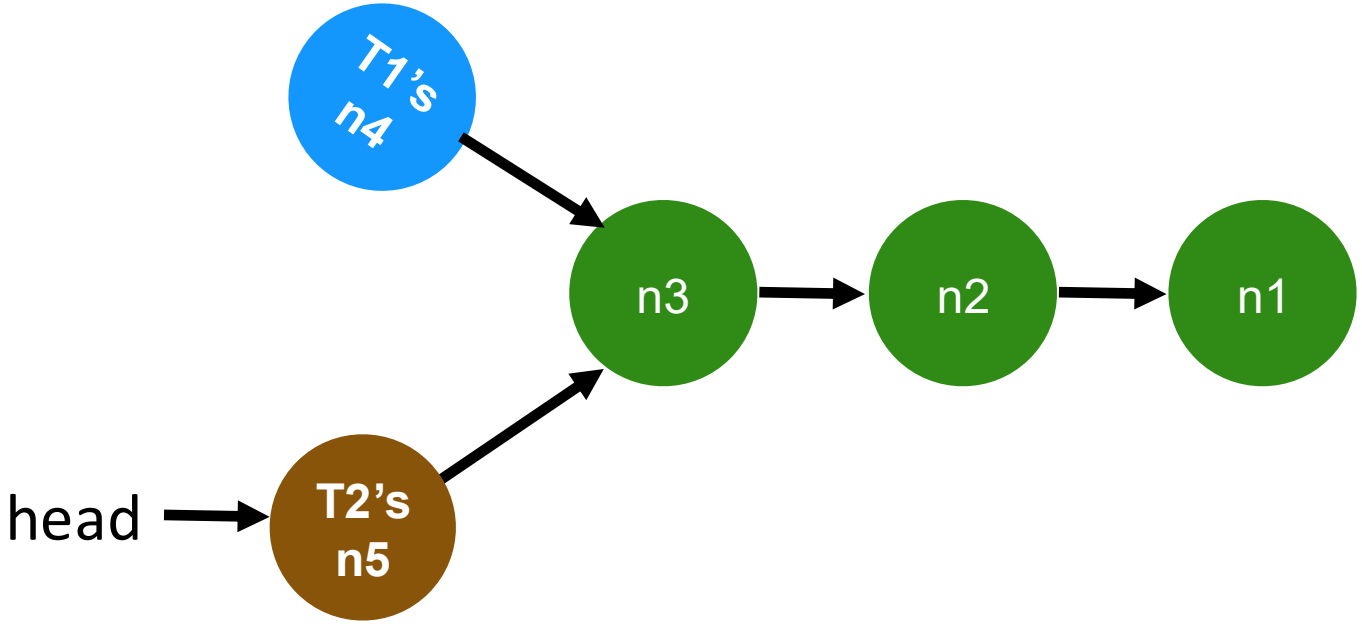
Thread 1	Thread 2
new->key = key	
new->next = L->head	

Linked-List Race



Linked-List Race

Thread 1	Thread 2
new->key = key	
new->next = L->head	
Cntxt_Switch()	new->key = key
	new->next = L->head
	L->head = new


T1's
n4
n3
n2
n1
head
T2's
n5

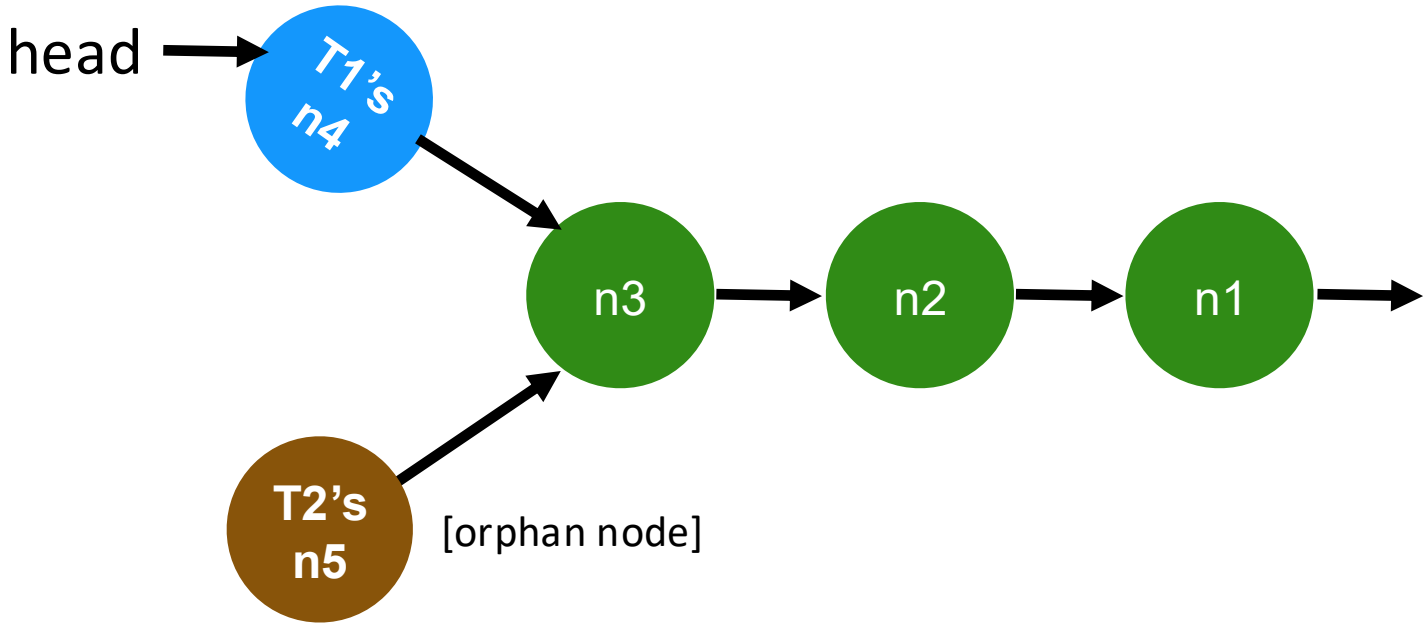
Linked-List Race

Thread 1	Thread 2
new->key = key	
new->next = L->head	
Cntxt_Switch()	new->key = key
	new->next = L->head
	L->head = new
	Cntxt_Switch()
L->head = new	

Both entries link directly to old head node

Only one entry (which one?) can be the new head.

Resulting Linked List



Locking Linked Lists

```
Void List_Insert(list_t *L, int key) {
        node_t *new =
                malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L, int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)
                                return
1;
                tmp = tmp->next;
        }
    return 0;
}
```

```
typedef struct __node_t {
        int key;
        struct __node_t *next;
} node_t;

Typedef struct __list_t {
        node_t *head;
} list_t;

Void List_Init(list_t *L) {
        L->head = NULL;
}
```

How to add locks?

Locking Linked Lists

```
typedef struct __node_t {
        int key;
        struct __node_t *next;
} node_t;

Typedef struct __list_t {
        node_t *head;
} list_t;

Void List_Init(list_t *L) {
        L->head = NULL;
}
```

How to add locks?

pthread_mutex_t lock;

One lock for entire list

```
typedef struct __node_t {
        int key;
        struct __node_t *next;
} node_t;

Typedef struct __list_t {
        node_t *head;
        pthread_mutex_t lock;
} list_t;

Void List_Init(list_t *L) {
        L->head = NULL;
        pthread_mutex_init(&L->lock, NULL);
}
```

Coarse-grained locking

Locking Linked Lists : Approach #1

Consider everything critical section

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

```
Void List_Insert(list_t *L,  int key) {

    node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L, int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)

                return 1;
                tmp = tmp->next;
        }
    return 0;
}
```

Locking Linked Lists : Approach #2

Can critical section be smaller?

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

```
Void List_Insert(list_t *L,  int key) {

    node_t *new = malloc(sizeof(node_t));
    assert(new);
    new->key = key;
    new->next = L->head;
    L->head = new;
}

int List_Lookup(list_t *L, int key) {
    node_t *tmp = L->head;
    while (tmp) {
        if (tmp->key == key)

            return 1;
        tmp = tmp->next;
    }
    return 0;
}
```

Locking Linked Lists : Approach #2

Can critical section be smaller?

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

Pthread_mutex_lock(&L->lock);

Pthread_mutex_unlock(&L->lock);

```
Void List_Insert(list_t *L,  int key) {

  node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L, int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)

                return 1;
                tmp = tmp->next;
        }
      return 0;
}
```

Locking Linked Lists : Approach #3

Can critical section be smaller?

Pthread_mutex_lock(&L->lock);

Finer-grained locking (still on entire list) permits more concurrency. However, must be careful.

Pthread_mutex_unlock(&L->lock);

~~Pthread_mutex_lock(&L->lock);~~

If no List_Delete(), locks not needed
(Any non-head element cannot change once inserted;
assume store to L->head is atomic)

~~Pthread_mutex_unlock(&L->lock);~~

```
Void List_Insert(list_t *L,  int key) {

  node_t *new = malloc(sizeof(node_t));
        assert(new);
        new->key = key;
        new->next = L->head;
        L->head = new;
}

int List_Lookup(list_t *L, int key) {
        node_t *tmp = L->head;
        while (tmp) {
                if (tmp->key == key)

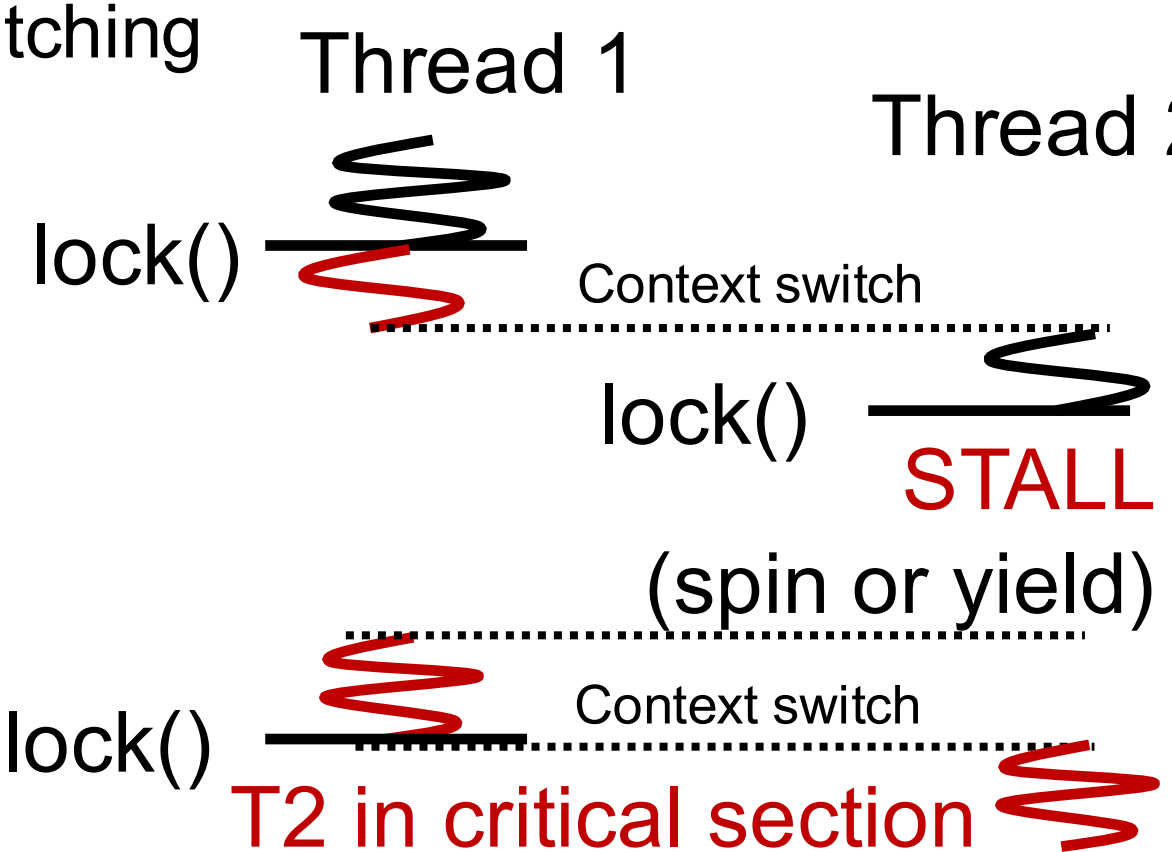
                return 1;
                tmp = tmp->next;
        }
    return 0;
}
```

Locking != No interruption

- In general, locks do not prevent context switching
- Locks prevent interleaving of critical section instructions across threads even under context switching

```
Void List_Insert(list_t *L,  int key) {
    node_t *new = malloc(sizeof(node_t));
    assert(new);
    new->key = key;
    lock(&l);
    new->next = L->head;
    L->head = new;
    unlock(&l);
}
```



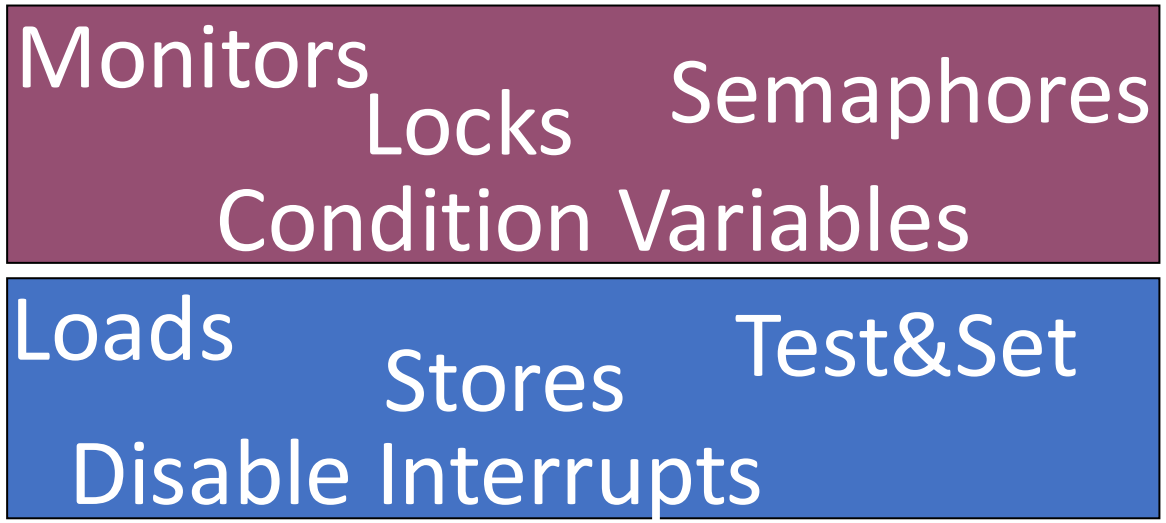


Implementing Synchronization

Build higher-level synchronization primitives in OS

- Operations that ensure correct ordering of instructions across threads

Build them once and get them right



Lock Implementation Goals

Correctness

- Mutual exclusion
  - Only one thread in critical section at a time
- Progress (deadlock-free)
  - If several simultaneous requests, must allow one to proceed

Fairness

- Bounded (starvation-free)
  - Must eventually allow each waiting thread to enter
- Threads wait roughly same time when locking repeatedly

Performance

CPU is not used unnecessarily (e.g., spinning)

Implementing LOCKS: w/ Load+Store

Code uses a single **shared** lock variable

```
Boolean flag = false; // shared variable
Void lock(Boolean *flag) {
     while (*flag) /* wait */ ;
     *flag = true;
}
Void unlock(Boolean *flag) {
     *flag = false;
}
```



Does this provide correct mutual exclusion?

Can you produce an example schedule that fails with 2 threads?

Race Condition with LOAD and STORE

```
*flag == 0 initially
```

Thread 1	Thread 2
`while(*flag == 1);`	
	`while(*flag== 1);`
	`*flag = 1`
`*flag = 1`	

Both threads grab lock!

Problem: Testing lock + setting lock is not atomic

An old solution: Peterson's algorithm

(assumes each load, store is atomic)

No longer applicable

(relaxed memory models: operations across threads not committed in order)

```
int flag[2];
int turn;

void init() {
    // indicate you intend to hold the lock w/ 'flag'
    flag[0] = flag[1] = 0;
    // whose turn is it? (thread 0 or 1)
    turn = 0;
}
void lock() {
    // 'self' is the thread ID of caller
    flag[self] = 1;
    // make it other thread's turn
    turn = 1 - self;
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // spin-wait while it's not your turn
}
void unlock() {
    // simply undo your intent
    flag[self] = 0;
}
```